



Birzeit University

Faculty of Engineering and Technology
Electrical and Computer Engineering Department

Course Project Report

ENCS3310 – Advanced Digital Design

Fast Multiplier Design

Stage 1: Ripple Carry Adder Implementation

Student Name: Karim Taha

Student ID: 1211155

Instructor: Dr. Abdellatif Abu-Issa

Course: ENCS3310 – Advanced Digital Design

Date: June 13, 2026

Abstract

A multiplier was designed in Verilog HDL using basic gate primitives. In Stage 1, the multiplier was built using full adders and ripple carry adders. In Stage 2, the ripple carry adders were replaced with carry look-ahead adders to improve the speed.

The correct operation was confirmed, and an intentional error was successfully detected by the testbench.

Contents

1	Design Overview	1
1.1	Gate Library (Student ID last digit = 5)	1
2	Hand Calculation	1
2.1	Maximum Clock Frequency	1
3	Simulation Results & Verification	2
3.1	Full Adder Propagation Delay – Waveform Analysis	2
3.2	Ripple Carry Adder Waveform	2
3.3	Multiplier Waveform	2
3.4	Intentional Error Injection	2
4	Stage 2 – Carry Look-Ahead Adder	3
4.1	CLA Adder Waveform	3
4.2	CLA Multiplier Waveform	3
4.3	CLA Multiplier Propagation Delay	3
4.4	CLA Intentional Error Injection	4
5	RCA vs CLA Comparison	4

Design Overview

The multiplier accepts a 4-bit input $\mathbf{X}$ and an n -bit generic input $\mathbf{Y}$, producing an $(n+4)$ -bit result $\mathbf{res}$, with all ports registered through D flip-flops on both input and output sides.

Gate Library (Student ID last digit = 5)

Gate	INV	NAND	NOR	AND	OR	XOR
Delay (ns)	2	4	4	6	6	9

Hand Calculation

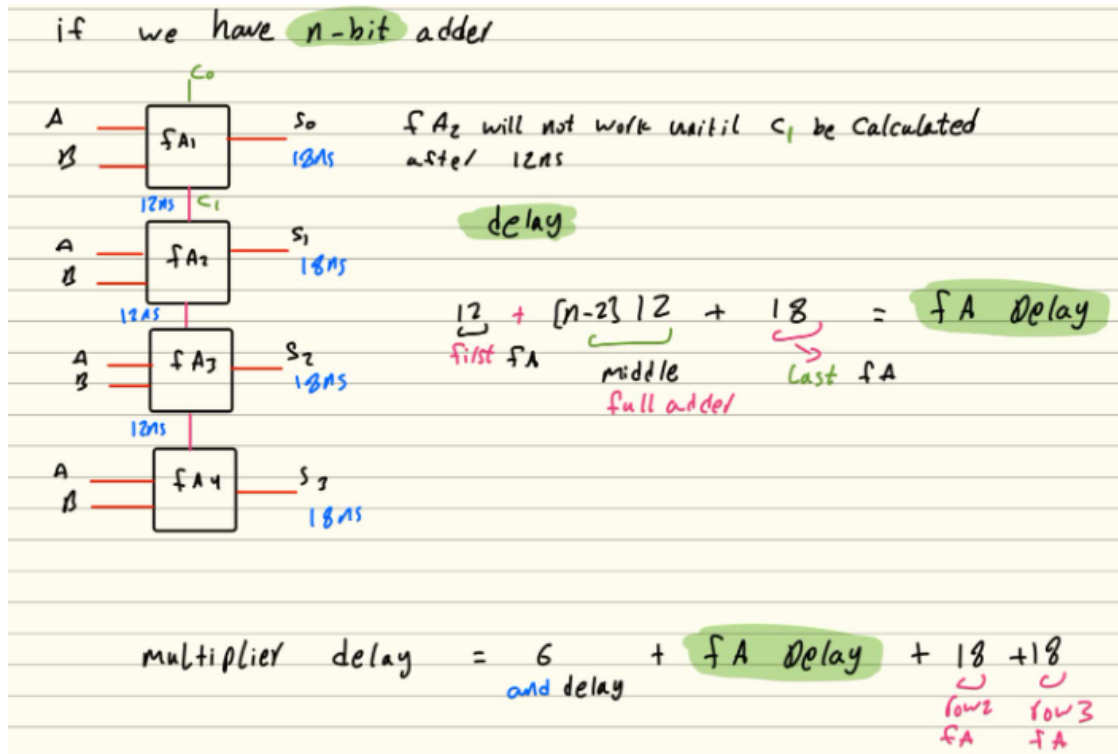


Figure 1: Hand-derived delay formulas for the full adder and n -bit RCA critical path.

Maximum Clock Frequency

$$f_{\max} = \frac{1}{T_{\text{clk}}} = \frac{1}{2t_{\text{mult}}}$$

Config.	n (bits)	t_{mult} (ns)	$T_{\text{clk}} = 2t_{\text{mult}}$ (ns)	$f_{\max}$ (MHz)
4×4	4	96	192	≈ 5.21
4×8	8	144	288	≈ 3.47
4×12	12	192	384	≈ 2.60

Frequency degrades linearly with n : each extra adder stage adds 54 ns of carry-ripple delay.

Simulation Results & Verification

Full Adder Propagation Delay – Waveform Analysis

When the input changes, the full adder needs time for the carry to ripple through the gates. A small glitch appears because signals travel through different paths with unequal delays as shown below.

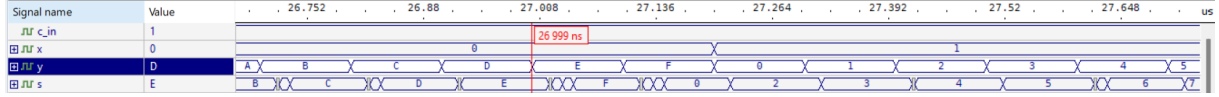


Figure 2: Full Adder Waveform 1

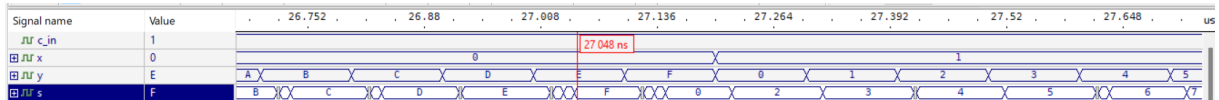


Figure 3: Full Adder Waveform 2

This waveform shows the output after the carry propagation is complete. The glitch has disappeared and the sum output has reached its correct value. Depending on the input pattern, the propagation delay can be around 27,ns and may reach up to 48,ns.

Ripple Carry Adder Waveform

The ripple carry adder adds bit by bit, and each carry moves to the next bit. For example, $0 + D + C = 19_{16}$, so the sum is 9 and the carry-out is 1.



Figure 4: 4-bit RCA waveform

Multiplier Waveform

At cursor $t = 34,409,705$ ns: $X = F_{16} = 15$, $Y = FF7_{16} = 4087$, result = $EF6A_{16} = 61,290$ ✓

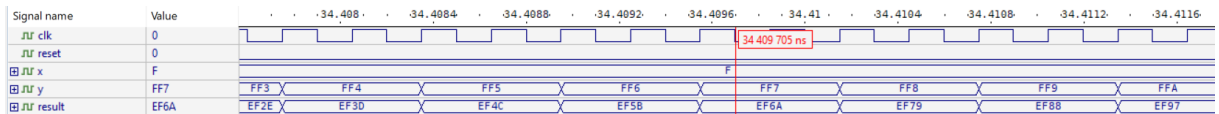


Figure 5: RCA multiplier swaveform

Intentional Error Injection

To validate the testbench, the following line in `multiplier.v` was deliberately changed:

Original (correct)	Modified (error)
<code>xy_shifted[1] = {{3{1'b0}}, yx[1], 1'b0};</code>	<code>xy_shifted[1] = {{4{1'b0}}, yx[1]};</code>

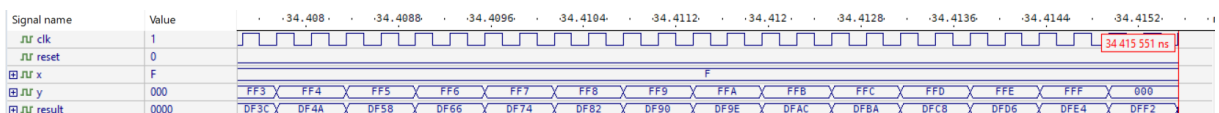


Figure 6: RAC multipller after error reported FAIL; only trivial zero-product cases passed.

```

° # KERNEL: 15 x 4086 | 57204 | 61290 | FAIL
° # KERNEL: 15 x 4087 | 57218 | 61305 | FAIL
° # KERNEL: 15 x 4088 | 57232 | 61320 | FAIL
° # KERNEL: 15 x 4089 | 57246 | 61335 | FAIL
° # KERNEL: 15 x 4090 | 57260 | 61350 | FAIL
° # KERNEL: 15 x 4091 | 57274 | 61365 | FAIL
° # KERNEL: 15 x 4092 | 57288 | 61380 | FAIL
° # KERNEL: 15 x 4093 | 57302 | 61395 | FAIL
° # KERNEL: 15 x 4094 | 57316 | 61410 | FAIL
° # KERNEL: 15 x 4095 | 57330 | 61425 | FAIL
° # KERNEL: 15 x 4096 | 0 | 61440 | FAIL
° # KERNEL: -----
° # KERNEL: PASS: 12299 / 65536
° # KERNEL: FAIL: 45059 / 65536

```

Figure 7: Waveform with injected error

Stage 2 – Carry Look-Ahead Adder

CLA Adder Waveform

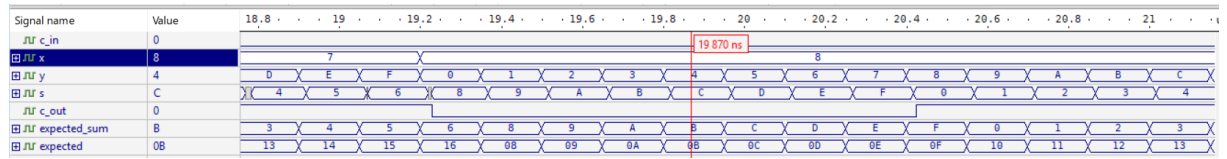


Figure 8: 4-bit CLA adder waveform.

The CLA adder computes all carry signals in parallel, so no carry ripple occurs. For example, when $x = 8$ and $y = 4$, $s = C_{16} = 12$ and $C_{out} = 0$, confirming that $8 + 4 = 12$.

CLA Multiplier Waveform

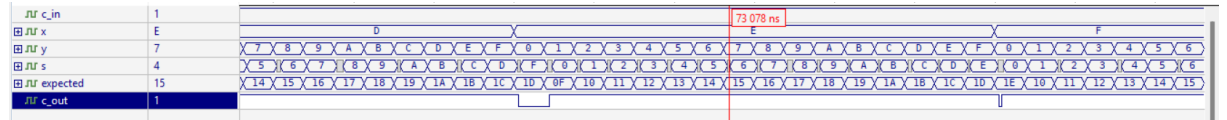


Figure 9: CLA multiplier simulation waveform.

The CLA multiplier replaces each RCA block with a CLA block, reducing the carry-ripple delay. For example, when $X = E_{16} = 14$ and $Y = 7$, the output is $\text{result} = 62_{16} = 98$, confirming that $14 \times 7 = 98$.

CLA Multiplier Propagation Delay

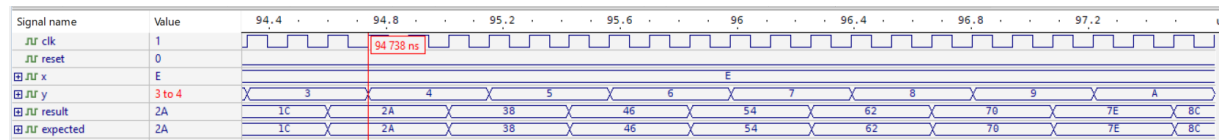


Figure 10: CLA multiplier delay – input changing at 94.738 ns, output not yet updated.

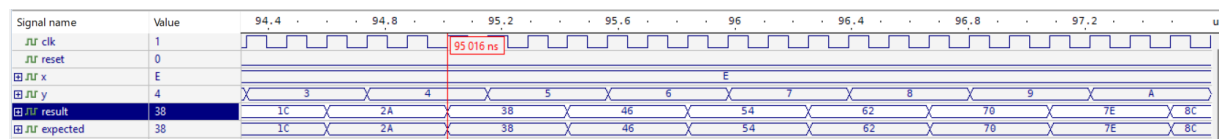


Figure 11: CLA multiplier delay

The measured delay is **278 ns**, equal to 2 registered clock cycles. For example, when $X = E_{16} = 14$ and y changes from $3 \rightarrow 4$, the output settles to $38_{16} = 56$, confirming $14 \times 4 = 56$).

CLA Intentional Error Injection

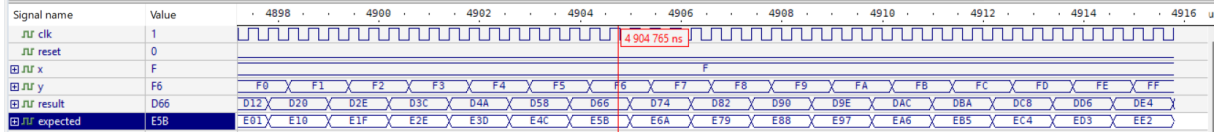


Figure 12: CLA multiplier waveform after error injection

$X = F_{16} = 15$, $Y = F6_{16}$): result shows D66 instead of expected E5B, confirming the misaligned partial product. Reverting the shift restored all cases to PASS.

RCA vs CLA Comparison

Property	Stage 1 – RCA	Stage 2 – CLA
Carry mechanism	Ripple (serial)	Look-ahead (parallel)
Carry delay	Grows with n	Fixed 2 gate levels
$t_{\text{mult}} (n = 4)$	96 ns	69 ns
$T_{\text{clk}} (n = 4)$	192 ns	138 ns
$f_{\text{max}} (n = 4)$	≈ 5.21 MHz	≈ 7.25 MHz
Speed improvement	–	$\approx 1.4\times$ faster
Glitches / hazards	Yes (ripple causes glitches)	Fewer (carry settled fast)
Scalability	Poor (slow at large n)	Better for small n blocks

Conclusion

A structural 4-bit $\times$ n -bit multiplier was successfully designed and verified using gate primitives. Critical-path analysis gives $t_{\text{mult}} = 96$ ns, $T_{\text{clk}} = 192$ ns, and $f_{\text{max}} \approx 5.21$ MHz for the 4×4 case, degrading linearly with n due to carry-ripple growth. Simulation confirmed correct results; error-injection testing proved the testbench reliably detects partial-product misalignment faults.